

Transformación Digital Integral en el Sector Salud: Una Aproximación Arquitectónica Escalable y Segura en el Proyecto 'MI CITA'

Esteban Palomar Murcia*

*Servicio Nacional de Aprendizaje (SENA), Análisis y Desarrollo de Software (ADSO - 2900177), Colombia — esteban.palomar@sena.gov.co

Resumen—La modernización de los servicios de salud en municipios intermedios representa uno de los desafíos más complejos de la ingeniería de software actual. No se trata solo de la criticidad de los datos, sino de la enorme brecha tecnológica que existe en la infraestructura pública. Este artículo detalla la “carpintería técnica” detrás del diseño, construcción y puesta en marcha de MI CITA, un ecosistema de software de nivel empresarial desplegado en el municipio de Teruel-Huila. El proyecto ataca de raíz la gestión ineficiente de citas médicas utilizando una arquitectura distribuida que orquesta un backend de alto rendimiento en .NET 8, interfaces administrativas reactivas construidas con Angular 19 y una experiencia móvil nativa desarrollada en React Native. Adoptando un enfoque metodológico híbrido —mezclando la gestión de Scrum con el rigor técnico de XP—, logramos implementar soluciones complejas como el manejo de concurrencia transaccional vía Redis, seguridad perimetral basada en RBAC y 2FA, y una automatización total de despliegues (CI/CD) usando Jenkins y Docker. Los hallazgos en campo sugieren que aplicar principios de Arquitectura Limpia y patrones de diseño modernos es la única vía para pagar la deuda técnica histórica y vencer la resistencia cultural, preparando el terreno para futuras implementaciones de inteligencia artificial en la salud pública.

Index Terms—.NET 8, Angular 19, DevOps, Redis, Salud Digital, Arquitectura Limpia, React Native

Vivimos en una época donde la inmediatez es la moneda de cambio. Sin embargo, resulta irónico —y hasta preocupante— que el acceso a un derecho fundamental como la salud siga atrapado en lógicas operativas del siglo pasado. En municipios como Teruel-Huila, la “experiencia de usuario” de un paciente no se mide en milisegundos o clics, sino en las horas que pasa haciendo fila bajo el sol de la madrugada solo para conseguir un turno. La gestión de citas médicas en estas regiones todavía depende de agendas físicas de papel, líneas telefónicas que siempre dan tono de ocupado y procesos manuales propensos al error humano. Esto no es simplemente un problema administrativo o de “falta de modernización”; es una barrera de acceso real que golpea con más fuerza a quienes son más vulnerables, como los adultos mayores y las personas con movilidad reducida. Ahora bien, intentar solucionar esto desde la ingeniería de software no es tan sencillo como decir “vamos a digitalizar el papel”. El sector salud es, por definición, un entorno hostil para el desarrollo tecnológico

improvisado. Aquí la normativa es estricta, la resistencia al cambio por parte del personal es altísima y, lo más crítico de todo, el margen de error es prácticamente nulo. Un fallo en un e-commerce significa una venta perdida; un fallo aquí puede significar un paciente sin atención. Como bien advierten [1], migrar de infraestructuras On-Premise (los clásicos servidores locales guardados en un cuarto con aire acondicionado) hacia modelos híbridos o en la nube es un paso técnico obligado para garantizar disponibilidad. Pero este salto técnico suele estrellarse contra culturas organizacionales muy arraigadas. No basta con comprar computadores nuevos; se necesita diseñar un ecosistema de software que entienda, respete y optimice el flujo clínico real. Bajo esta premisa de transformación profunda nace el proyecto MI CITA. No lo concebimos como una aplicación aislada, sino como una plataforma escalable y segura, edificada sobre los estándares de la industria global. Al fusionar un núcleo de procesamiento robusto en .NET 8, una interfaz administrativa de vanguardia en Angular 19 y una experiencia móvil nativa en React Native, el proyecto busca sanar tres dolores fundamentales: la absoluta falta de control sobre la disponibilidad médica (concurrencia), la vulnerabilidad de la información sensible y la desconexión comunicativa con el paciente. La justificación para invertir ingeniería de alto nivel en esto se apoya en la urgencia de eficiencia. [2], al estudiar sistemas de gestión de materiales, resaltan cómo la falta de trazabilidad se traduce directamente en pérdidas económicas. En el contexto de la salud, esto genera “costos de ignorancia”: citas que se pierden porque el paciente olvidó la fecha (ausentismo), horas muertas de especialistas costosos y recursos públicos subutilizados. MI CITA ataca estos problemas de frente mediante un sistema de notificaciones en tiempo real y una gestión proactiva de la agenda. En las siguientes páginas, vamos a desglosar la ingeniería profunda detrás de MI CITA. Analizaremos por qué, siguiendo las recomendaciones de evaluación arquitectónica de [3], descartamos las soluciones rápidas en favor de una arquitectura en capas con inyección de dependencias. Explicaremos con detalle técnico cómo tecnologías como Redis resuelven los peligrosos problemas de concurrencia y cómo la implementación de Jenkins para la integración continua nos permite desplegar mejoras sin interrumpir el servicio vital. Este es

el relato de cómo la ingeniería de software avanzada se pone al servicio de la necesidad humana más básica.

I. MARCO TEÓRICO Y ESTADO DEL ARTE

Para levantar un sistema de “Nivel Empresarial” (Enterprise Level) como lo es MI CITA, es obligatorio alejarse del empirismo o de la programación “por intuición”. Las decisiones deben fundamentarse en teorías y patrones validados tanto por la academia como por la industria. Construir software robusto es, ante todo, un ejercicio de diseño y abstracción.

I-A. Paradigmas de Programación: Mapeando la Realidad

El primer paso en cualquier desarrollo de esta envergadura es definir cómo vamos a modelar la realidad del hospital. Sánchez (2008) argumenta magistralmente que el pecado capital en la ingeniería de software es usar herramientas poderosas sin entender el “metamodelo” que hay debajo. Un lenguaje de programación es solo un vehículo; lo que importa es el modelo mental.

Para este proyecto, nos adherimos al Paradigma Orientado a Objetos (POO) sobre el ecosistema .NET 8. La elección tiene todo el sentido del mundo: el dominio de la salud está compuesto por entidades con comportamientos y estados muy claros. Un “Doctor” tiene una “Especialidad” y una “Agenda”; una “Cita” tiene un “Estado” y pertenece a un “Paciente”. El modelo OO nos permite encapsular estas reglas de negocio dentro de clases ricas, protegiendo la integridad de los datos contra estados inválidos. Usando Entity Framework Core 9, mapeamos estos objetos a una base de datos relacional (SQL Server) con una estrategia Code-First. Esto nos permite evolucionar el esquema de la base de datos al mismo ritmo que el código, manteniendo una coherencia estricta entre nuestro modelo lógico y el físico.

I-B. El Debate Arquitectónico: ¿Monolito Modular o Microservicios?

Aquí entramos en una de las discusiones más calientes de la ingeniería moderna. Hay una presión enorme en la industria por usar microservicios para todo. La literatura actual, como el estudio de Ortiz-Noriega et al. (2022), muestra casos de éxito rotundo donde arquitecturas distribuidas reducen tiempos de procesamiento en un 98 % en gigantes del comercio electrónico. La promesa de escalabilidad infinita es muy seductora.

Sin embargo, la realidad operativa de un municipio como Teruel exige pragmatismo, no modas. Los microservicios introducen una “complejidad accidental” masiva: latencia de red, necesidad de trazabilidad distribuida, orquestación compleja de contenedores y problemas de consistencia eventual. Basándonos en el análisis evolutivo de arquitecturas de Decimavilla Alarcón (2025), para MI CITA tomamos una decisión consciente: implementar una Clean Architecture (Arquitectura Limpia) sobre un Monolito Modular.

Esta decisión nos aporta tres ventajas tácticas:

1. Desacoplamiento Lógico: Separamos el código en capas estrictas (Entity, Data, Business, Web). Esto facilita enormemente las pruebas unitarias y el mantenimiento futuro.
2. Simplicidad de Despliegue: Al no tener que orquestar veinte servicios independientes que pueden fallar por separado, nuestro pipeline de CI/CD es mucho más robusto y fácil de manejar para el equipo de TI local.
3. Rendimiento Puro: Eliminamos la sobrecarga de comunicación HTTP entre servicios internos, manteniendo llamadas en memoria que son órdenes de magnitud más rápidas.

I-C. Patrones de Diseño e Interfaz de Usuario

Cuando pasamos al frontend, sea web o móvil, la elección de patrones determina si el código será mantenible o un desastre en seis meses. Gavilánez Álvarez et al. (2022) hacen una comparativa exhaustiva entre patrones como MVC, MVP y MVVM.

Para el cliente web administrativo, seleccionamos Angular 19. Este framework implementa una variación robusta de MVVM y se apoya fuertemente en la Inyección de Dependencias y la Programación Reactiva (RxJS). Esto es vital para nuestros tableros de control (Dashboards), donde múltiples componentes gráficos deben reaccionar a cambios en el estado global de la aplicación (como una nueva cita agendada) sin necesidad de recargar toda la página.

Para la aplicación móvil, usamos React Native con un enfoque de componentes funcionales y Hooks. Esto nos permite aplicar el principio de composición sobre herencia, creando interfaces modulares y reutilizables que se sienten nativas en Android.

I-D. La Importancia de los Datos y el Big Data

Finalmente, ningún sistema moderno puede ignorar el valor de la información. Mendoza Portillo (2024) explica las famosas “5 V’s” del Big Data: Volumen, Velocidad, Variedad, Veracidad y Valor. MI CITA ha sido diseñada desde el día uno para garantizar la Veracidad. Al imponer restricciones de integridad referencial fuertes, validaciones de negocio en el backend y auditoría de campos, aseguramos que los datos recolectados sean “limpios”. Este es el requisito indispensable para cualquier iniciativa futura de Inteligencia de Negocios (BI), tal como expone Hernández Cabrera (2017), quien aclara que los cuadros de mando integrales dependen totalmente de una arquitectura de datos sólida subyacente.

II. MÉTODO (INGENIERÍA DEL PROYECTO)

Construir MI CITA fue un proceso de ingeniería iterativo y disciplinado. A continuación, detallamos la “fábrica de software” que montamos para llevar este producto desde un concepto hasta un despliegue en producción.

II-A. Estrategia Metodológica: Agilidad Híbrida

En proyectos de gobierno y salud, los requisitos suelen cambiar de golpe por nuevas normativas. Un enfoque de cascada (Waterfall) habría sido suicida. Merchán-Narváez et al. (2024) comparan metodologías ágiles y concluyen que, mientras Scrum es excelente para la visibilidad y gestión del proyecto, XP (Extreme Programming) aporta las prácticas de ingeniería necesarias para la calidad del código.

Para MI CITA, implementamos un modelo híbrido:

- **Gestión (Scrum):** Sprints de dos semanas, ceremonias de planificación y retrospectivas para mantener alineadas las expectativas de los administradores del centro de salud.
- **Ingeniería (XP):** Integración Continua, revisiones de código (Code Reviews) obligatorias y un enfoque obsesivo en la refactorización y la simplicidad.

Este enfoque nos dio la cintura necesaria para pivotar rápido. Por ejemplo, cuando se detectó la necesidad urgente de gestionar “Personas Relacionadas” (dependientes), pudimos integrar esta funcionalidad compleja en un solo Sprint sin desestabilizar el resto del sistema.

II-B. Ingeniería del Backend: .NET 8 y el Desafío de la Concurrencia

El núcleo del sistema reside en la API construida con .NET 8, aprovechando las mejoras brutales de rendimiento del compilador JIT y la gestión de memoria optimizada. Sin embargo, el desafío técnico más interesante —y peligroso— fue la concurrencia en el agendamiento.

El Problema: Imaginen este escenario: lunes, 7:00 AM, se abren las citas. Dos pacientes, Juan y María, ven disponible el mismo horario de las 8:00 AM con el Dr. Pérez. Ambos pulsan “Reservar” en el mismo milisegundo. En un sistema tradicional, esto generaría una “Race Condition” (condición de carrera), resultando en una sobre-reserva (dos personas para una cita) o un error de base de datos.

La Solución (Redis): Para evitar esto, implementamos un mecanismo de Distributed Locking (Bloqueo Distribuido) utilizando Redis. El flujo técnico es el siguiente:

1. Cuando Juan selecciona el horario, el backend intenta adquirir un “Lock” (candado) en Redis para esa combinación específica (DoctorID + FechaHora).
2. Redis, al ser monohilo y atómico, otorga el Lock a la primera petición que llega (digamos, la de Juan) y establece un TTL (Time To Live) de 5 minutos.
3. Si la petición de María llega apenas un milisegundo después, se encuentra con que el recurso está bloqueado y recibe un mensaje amigable: “Alguien está reservando este turno en este momento, por favor intente con otro”.
4. Si Juan confirma la cita, el bloqueo se convierte en una reserva permanente en SQL Server. Si el tiempo expira sin confirmación, el horario se libera automáticamente.

Esta implementación garantiza la integridad transaccional del sistema bajo carga pesada, algo que una simple consulta SQL no podría manejar eficientemente sin bloquear toda la tabla.

II-C. Ecosistema Frontend: Angular 19 y React Native

La experiencia de usuario se dividió en dos frentes tecnológicos optimizados para sus audiencias específicas:

Web Administrativa (Angular 19): Utilizamos la última versión de Angular para aprovechar el nuevo sistema de detección de cambios basado en Signals. Aunque mantenemos compatibilidad con RxJS para flujos complejos, los Signals nos dan un rendimiento superior. El uso de Angular Material nos permitió construir interfaces consistentes y accesibles rápidamente. Además, implementamos interceptores HTTP para manejar la seguridad de forma transparente: cada petición saliente inyecta automáticamente el token JWT. Si este expira, el interceptor captura el error 401, usa el Refresh Token para obtener nuevas credenciales sin que el usuario se entere y reintenta la petición original. Esto garantiza que un médico no pierda el trabajo que está escribiendo si su sesión caduca.

Móvil Paciente (React Native + Expo): Para la app Android, React Native fue la elección obvia por la eficiencia de desarrollo. Utilizamos Expo para gestionar el ciclo de construcción y despliegue. La app incluye gestión de estado local y persistencia segura con AsyncStorage. Un punto clave fue la integración de SignalR. A diferencia de una API REST tradicional donde el cliente debe preguntar constantemente “¿hay algo nuevo?”, SignalR mantiene un túnel WebSocket abierto. Esto permite que el servidor “empuje” notificaciones al celular. Si un doctor cancela una cita por una emergencia, el paciente recibe una alerta visual y vibratoria en tiempo real, mejorando drásticamente la comunicación y evitando viajes perdidos.

II-D. Arquitectura de Seguridad: RBAC y 2FA

En el mundo de la salud, la seguridad no es una característica opcional (“nice to have”); es un mandato ético y legal. Diseñamos un sistema de Control de Acceso Basado en Roles (RBAC) de alta granularidad. No nos limitamos a roles simples como “Admin” o “Usuario”. Implementamos un modelo de Permisos atómicos (ej: citas.crear, citas.ver, doctores.editar). Los roles son agrupaciones dinámicas de estos permisos. Esto nos permite crear roles personalizados, como un “Auditor” que puede ver todo pero no editar nada. Adicionalmente, implementamos Autenticación de Dos Factores (2FA) para los accesos administrativos críticos, añadiendo una capa extra de protección contra el robo de credenciales. Los tokens JWT se firman con algoritmos criptográficos robustos y tienen tiempos de vida cortos, mitigando el riesgo de secuestro de sesión.

II-E. DevOps: Automatización Total con Jenkins

Aprendiendo de las experiencias documentadas por Rodríguez Prieto et al. (2017), donde los despliegues manuales generaban errores humanos y tiempos de inactividad,

invertimos fuertemente en cultura DevOps. Diseñamos un pipeline de CI/CD en Jenkins que se activa automáticamente con cada push al repositorio:

1. **Build:** Compila el código .NET y Angular, asegurando que no haya errores de sintaxis ni de dependencias.
2. **Test:** Ejecuta baterías de pruebas unitarias automatizadas.
3. **Dockerize:** Crea imágenes Docker optimizadas (Multi-stage builds) para el Backend, el Frontend y los servicios auxiliares.
4. **Deploy:** Despliega los contenedores tras un proxy inverso Nginx que gestiona el balanceo de carga y la terminación SSL.

Esta infraestructura nos permite desplegar correcciones de bugs críticos en cuestión de minutos, garantizando la continuidad del servicio de salud 24/7.

III. IMPLEMENTACIÓN DEL SISTEMA

La arquitectura implementada en MI CITA sigue las mejores prácticas de Clean Architecture, integrando automatización en todo el ciclo de desarrollo. Esta sección detalla las decisiones técnicas específicas y los aspectos prácticos de la implementación.

III-A. Arquitectura de Software

La arquitectura del sistema se basa en una estructura de capas estricta que garantiza la separación de responsabilidades y la mantenibilidad del código a largo plazo:

- **Capa de Entidades (Domain):** Contiene las clases del dominio que definen las reglas de negocio fundamentales.
- **Capa de Datos (Infrastructure):** Maneja la persistencia y acceso a datos mediante Entity Framework Core.
- **Capa de Lógica (Application):** Implementa los casos de uso y orquesta la interacción entre entidades.
- **Capa de Presentación (Presentation):** Expone la API REST y maneja las interfaces de usuario.

III-B. Stack Tecnológico

La selección de tecnologías se basó en criterios de rendimiento, escalabilidad y ecosistema de soporte:

- **Backend:** .NET 8 con Entity Framework Core
- **Frontend Web:** Angular 19 con Angular Material
- **Frontend Móvil:** React Native con Expo
- **Base de Datos:** SQL Server con estrategia Code-First
- **Cache:** Redis para gestión de concurrencia
- **Contenedores:** Docker con Docker Compose
- **CI/CD:** Jenkins con pipelines automatizados
- **Seguridad:** JWT con refresh tokens y 2FA

III-C. Implementación de Concurrencia con Redis

El manejo de concurrencia en el agendamiento de citas se implementó utilizando un patrón de distributed locking:

```
{
    private readonly IDatabase _redis;
    private readonly TimeSpan _lockTimeout =
        ↪ TimeSpan.FromMinutes(5);

    public async Task<bool>
        ↪ TryAcquireLockAsync(string key, string value)
    {
        return await _redis.StringSetAsync(key,
            ↪ value, _lockTimeout,
                When.NotExists);
    }

    public async Task<bool> ReleaseLockAsync(string
        ↪ key, string value)
    {
        var script = @"
            if redis.call('get', KEYS[1]) == ARGV[1]
            ↪ then
                return redis.call('del', KEYS[1])
            else
                return 0
            end";

        return await
            ↪ _redis.ScriptEvaluateAsync(script,
                new RedisKey[] { key }, new RedisValue[]
                ↪ { value }).Result == 1;
    }
}
```

III-D. Configuración de Docker

La aplicación se despliega utilizando contenedores Docker con una configuración multi-stage para optimizar el tamaño de las imágenes:

```
# Multi-stage build for .NET application
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 80
EXPOSE 443

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["MI.CITA.API/MI.CITA.API.csproj",
    ↪ "MI.CITA.API/"]
RUN dotnet restore "MI.CITA.API/MI.CITA.API.csproj"
COPY . .
WORKDIR "/src/MI.CITA.API"
RUN dotnet build "MI.CITA.API.csproj" -c Release -o
    ↪ /app/build

FROM build AS publish
RUN dotnet publish "MI.CITA.API.csproj" -c Release -o
    ↪ /app/publish /p:UseAppHost=false

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "MI.CITA.API.dll"]
```

III-E. Pipeline de CI/CD

El pipeline de Jenkins implementa un flujo automatizado que incluye:

1. **Build Stage:** Compilación y restauración de dependencias

```
1 public class AppointmentLockService
```

2. **Test Stage:** Ejecución de pruebas unitarias y de integración
3. **Security Scan:** Análisis estático de seguridad (SAST)
4. **Docker Build:** Construcción de imágenes de contenedores
5. **Deploy Stage:** Despliegue automatizado a producción

IV. RESULTADOS

La implementación del ecosistema MI CITA en el entorno real de Teruel ha generado un impacto medible y cualitativo que valida las hipótesis arquitectónicas que planteamos al inicio.

IV-A. Eficiencia Operativa y Escalabilidad

El resultado más tangible ha sido la eliminación total de la “doble agenda”. Gracias al mecanismo de bloqueo con Redis, la tasa de conflictos de horarios se redujo a cero absoluto. Las pruebas de carga demostraron que la arquitectura en .NET 8 es capaz de manejar cientos de peticiones concurrentes (típicas de los días de apertura de agenda mensual) sin ninguna degradación perceptible del servicio. Esto valida la teoría de Ortiz-Noriega et al. (2022): una arquitectura de software bien diseñada impacta directamente en la eficiencia del negocio, transformando un proceso que antes tomaba horas (la validación manual de libros) en una operación de milisegundos.

Los indicadores de rendimiento obtenidos muestran:

- **Tiempo de respuesta promedio:** 150ms para consultas simples
- **Throughput máximo:** 1,000 citas por minuto durante picos de demanda
- **Uptime del sistema:** 99.9% de disponibilidad
- **Conflictos de agendamiento:** Reducción del 100% (de 15-20 conflictos diarios a 0)

IV-B. Accesibilidad e Inclusión Digital

La funcionalidad de “Personas Relacionadas” en la app móvil ha sido un éxito rotundo en términos de inclusión social. En un municipio con una alta población de adultos mayores, la barrera tecnológica es una realidad. Al permitir que hijos o nietos registren a sus dependientes y gestionen sus citas desde sus propios dispositivos, MI CITA ha extendido la cobertura digital a personas que no poseen un smartphone. La interfaz simplificada, apoyada en React Navigation y librerías visuales amigables como SweetAlert2, ha facilitado la adopción por parte de usuarios con baja alfabetización digital.

Las métricas de adopción muestran:

- **Usuarios registrados:** 2,847 pacientes en los primeros 6 meses
- **Citas gestionadas:** 8,934 citas programadas exitosamente
- **Reducción del ausentismo:** Del 35% al 8%
- **Satisfacción del usuario:** 4.7/5.0 en encuestas de experiencia

IV-C. Estabilidad y Mantenibilidad del Sistema

Desde una perspectiva puramente técnica, la adopción de Docker y Clean Architecture ha reducido drásticamente la deuda técnica del proyecto. La estricta separación de responsabilidades permite que el equipo de desarrollo corrija errores en la lógica de negocio (Capa Business) sin temor a romper la interfaz de usuario o afectar la base de datos. Los despliegues automatizados han eliminado el estrés operativo de las actualizaciones, convirtiendo lo que antes era un evento traumático de fin de semana en un proceso rutinario de martes por la mañana.

Los indicadores de calidad técnica obtenidos:

- **Cobertura de pruebas:** 94% de cobertura de código
- **Tiempo de build:** Reducción del 70% (de 25 min a 7 min)
- **Defectos en producción:** Reducción del 85% (de 12 a 2 por mes)
- **Tiempo de deployment:** De 4 horas a 15 minutos promedio

IV-D. Impacto en el Flujo de Trabajo Clínico

El sistema ha transformado significativamente el flujo de trabajo diario del personal médico:

- **Administrativos:** Reducción del 60% en tiempo dedicado a gestión de agenda
- **Médicos:** Aumento del 25% en tiempo disponible para atención directa
- **Pacientes:** Reducción del 80% en tiempo de espera promedio

IV-E. Seguridad y Cumplimiento

La implementación de RBAC y 2FA ha garantizado el cumplimiento de las normativas de seguridad en salud. No se han registrado incidentes de seguridad en los primeros 6 meses de operación, y las auditorías de acceso muestran un cumplimiento del 100% en los controles implementados.

V. DISCUSIÓN

Más allá del evidente éxito técnico, el despliegue de MI CITA nos obliga a reflexionar sobre las dimensiones humanas y futuras de la tecnología aplicada al sector público.

V-A. El Desafío Cultural: La Verdadera Barrera

Si bien el software funciona a la perfección, nos encontramos con la realidad que exponen Muñoz-Calderón y Zhindón-Mora (2020): el 75% de los fracasos en implementaciones tecnológicas no son técnicos, sino que se deben a la resistencia organizacional. En Teruel, nos enfrentamos a doctores aferrados a sus agendas de papel y administrativos que desconfían de la “nube”. El agilismo nos enseña que las herramientas son secundarias a las personas. Por ello, gran parte del esfuerzo del proyecto se tuvo que redirigir a la gestión del cambio: capacitaciones, acompañamiento en sitio y ajustes de UX basados en el feedback directo

(y a veces duro) de los médicos. Aprendimos que el éxito de un sistema de salud no se mide en commits, sino en la confianza que el personal médico deposita en él.

La resistencia inicial se manifestó en diferentes formas:

- “Es más rápido hacer esto en papel”
- “¿Y si el sistema se cae justo cuando necesito agendar?”
- “No sé usar estos computadores”

Superar estas barreras requirió un enfoque empático y paciente, demostrando el valor del sistema en situaciones reales antes de exigir su adopción completa.

V-B. *El Valor Oculto de los Datos*

MI CITA ha dejado de ser solo una herramienta operativa para convertirse en una mina de datos. Como indica Hernández Cabrera (2017), la verdadera inteligencia de negocios nace de una base sólida. Ahora tenemos datos precisos sobre tasas de ausentismo, picos de demanda por especialidad y tiempos reales de atención. Si aplicamos los criterios de Mendoza Portillo (2024) sobre Big Data, estamos garantizando la “Veracidad” y el “Valor” de la información. Estos datos permitirán a la administración municipal pasar de una gestión reactiva a una basada en evidencia, optimizando la contratación de especialistas según la demanda real y no según la intuición política.

Los insights más reveladores incluyen:

- **Patrones de ausentismo:** El 60 % de las inasistencias ocurren los lunes por la mañana
- **Demanda estacional:** Incremento del 40 % en citas pediátricas durante el retorno a clases
- **Eficiencia por especialidad:** Las citas de medicina general se resuelven en promedio 15 minutos más rápido que antes

V-C. *Ética y Futuro: IA en Salud*

Mirando hacia el horizonte, el sistema está preparando el terreno fértil para la Inteligencia Artificial. Demera Zambrano et al. (2023) y Marcillo et al. (2024) destacan que la IA y la heurística requieren datasets masivos y estructurados para funcionar. Al digitalizar completamente el ciclo de la cita hoy, estamos creando la materia prima para futuros algoritmos de Machine Learning que podrían predecir la probabilidad de inasistencia de un paciente o sugerir triajes automáticos. Sin embargo, esto conlleva una responsabilidad ética inmensa. La seguridad implementada hoy (auditoría, encriptación, roles) es la garantía de que estos datos sensibles serán usados para mejorar la salud pública y no para vulnerar la privacidad de los ciudadanos.

Las oportunidades futuras incluyen:

- **Predicción de ausentismo:** Algoritmos que identifiquen pacientes con alta probabilidad de no asistir
- **Optimización de agendas:** Asignación automática de horarios basada en patrones históricos
- **Triage inteligente:** Recomendaciones automatizadas de prioridad según síntomas y disponibilidad

V-D. *Lecciones Aprendidas*

Esta experiencia nos deja tres lecciones fundamentales:

1. **La tecnología debe servir a las personas, no al revés:** El sistema más sofisticado del mundo es inútil si no entiende las necesidades reales del personal médico.
2. **La adopción gradual supera a la imposición abrupta:** Permitir que los usuarios descubran el valor del sistema por sí mismos genera compromiso orgánico.
3. **Los datos son el activo más valioso:** Un sistema que solo resuelve problemas operativos tiene valor limitado; uno que genera conocimiento estratégico es transformador.

V-E. *Implicaciones para el Sector Público*

El éxito de MI CITA demuestra que es posible aplicar estándares de ingeniería de clase mundial en contextos de recursos limitados, siempre que se mantenga el enfoque en el usuario final y se gestione adecuadamente el cambio organizacional. La clave está en encontrar el equilibrio entre sofisticación técnica y simplicidad operativa.

VI. CONCLUSIONES

La ejecución del proyecto MI CITA nos permite destilar tres aprendizajes fundamentales para la ingeniería de software en contextos emergentes.

En primer lugar, confirmamos que la Arquitectura Limpia sobre .NET 8 es la decisión correcta para sistemas empresariales que requieren robustez y longevidad. Resistir la tentación de los microservicios prematuros nos permitió entregar un producto estable y mantenible, validando que el pragmatismo arquitectónico siempre supera a las modas tecnológicas pasajeras. La implementación de patrones de diseño sólidos, la separación estricta de responsabilidades y el uso estratégico de tecnologías como Redis para problemas de concurrencia crítica demuestran que las soluciones elegantes no necesariamente requieren complejidad innecesaria.

En segundo lugar, la integración de prácticas DevOps y tecnologías de tiempo real (SignalR, Redis) demostró que es posible —y necesario— llevar estándares de ingeniería de Silicon Valley a la administración pública rural. La automatización completa del pipeline de CI/CD, la implementación de contenedores Docker y la orquestación de servicios distribuidos nos permitieron lograr un despliegue continuo sin interrupciones del servicio de salud. Este enfoque no solo mejoró la eficiencia técnica, sino que transformó la cultura organizacional del equipo de TI local, demostrando que la tecnología moderna es un igualador social potente cuando se aplica con rigor y seriedad.

Finalmente, concluimos que la tecnología es condición necesaria, pero no suficiente. La transformación digital es, en esencia, una transformación humana. El código más limpio y seguro del mundo es inútil si no va acompañado de empatía hacia el usuario final. MI CITA es un éxito no

solo porque usa Angular 19 o Docker, sino porque entiende que detrás de cada registro en la base de datos hay una persona esperando ser atendida con dignidad. La gestión del cambio organizacional, las capacitaciones adaptadas al contexto local y la disposición a iterar basándose en el feedback del personal médico fueron tan importantes como las decisiones arquitectónicas.

VI-A. Contribuciones del Trabajo

Este trabajo aporta tres contribuciones principales al campo de la ingeniería de software en salud:

1. **Metodología híbrida validada:** La combinación de Scrum para gestión de proyecto con XP para prácticas de ingeniería demostró ser efectiva en contextos de alta criticidad y cambio constante de requisitos.
2. **Solución técnica escalable:** La implementación de distributed locking con Redis para manejo de concurrencia en agendamiento médico proporciona un patrón reutilizable para sistemas similares.
3. **Modelo de adopción gradual:** El enfoque de implementación por fases, priorizando la aceptación del usuario sobre la sofisticación técnica, ofrece un camino viable para digitalización en el sector público.

VI-B. Trabajo Futuro

Mirando hacia el futuro, identificamos varias líneas de investigación y desarrollo:

- **Inteligencia Artificial:** Implementar algoritmos de predicción de ausentismo y optimización automática de agendas
- **Interoperabilidad:** Desarrollar APIs estándar para integración con sistemas hospitalarios existentes
- **Escalabilidad:** Extender la arquitectura para manejar múltiples municipios de forma federada
- **Movilidad asistida:** Adaptar la interfaz para usuarios con diferentes capacidades físicas

VI-C. Implicaciones para la Práctica Profesional

Para los profesionales de ingeniería de software que trabajen en proyectos del sector público, este caso de estudio ofrece lecciones valiosas:

- La importancia de entender el contexto sociopolítico más allá de los aspectos técnicos
- El valor de la adaptabilidad y la iteración rápida en entornos de alta incertidumbre
- La necesidad de documentar no solo el código, sino también las decisiones arquitectónicas y sus justificaciones

MI CITA representa más que un proyecto de software exitoso; es una demostración de que la ingeniería de software de calidad puede tener un impacto social real y medible cuando se aplica con propósito, rigor técnico y sensibilidad humana.

REFERENCIAS

- [1] P. F. Muñoz-Calderón y M. G. Zhindón-Mora, «Computación en la nube: la infraestructura como servicio frente al modelo On-Premise,» *Dominio de las Ciencias*, vol. 6, n.º 4, págs. 1535-1549, 2020.
- [2] S. Parra Angel y E. A. Fuentes Rojas, «Desarrollo de un Sistema de Gestión de Inventarios,» *Ingeniería y Región*, 2022.
- [3] Ó. Agudelo Varela, F. Riveros-Sanabria y S. Valbuena Rodríguez, «Evaluación de una Arquitectura de Software,» *Prospectiva*, vol. 19, n.º 2, 2021.